



Q3 • 2021

ThinkstScapes Quarterly

research@thinkst.com

<https://thinkst.com/ts>

Brought to you by

**Most companies find out way too late
that they've been breached.**

Thinkst Canary changes this.

Canaries deploy in under 4 minutes
and require 0 ongoing admin overhead.

They remain silent till they need to chirp,
and then, you receive that single alert.

When.it.matters.

Find out why some of the smartest security teams
in the world swear by Thinkst Canary.

<https://canary.love>

Contents

Introduction	4
Themes covered in this issue	6
Into the embedded realm	7
Precursor: Towards Evidence-Based Trust in Hardware	8
Kernel Pwning with eBPF: a Love Story	9
InternalBlue / Frankenstein / Spectra	10
HALucinator: Firmware Re-hosting Through Abstraction Layer Emulation	11
Device-agnostic Firmware Execution is Possible: A Concolic Execution Approach for Peripheral Emulation	11
Remote Timing Attacks on TPMs, AKA TPM-Fail	12
Breaking VSM by Attacking SecureKernel	13
Whispers Among the Stars: Perpetrating (and Preventing) Satellite Eavesdropping Attacks	14
Exploiting 'differences of opinion'	15
HTTP/2: The Sequel is Always Worse	16
Differential Fuzzing of x86-64 Instruction Decoders	16
EtherOops: Exploring Practical Methods to Exploit Ethernet Packet-in-Packet Attacks	17
Light Commands: Laser-Based Audio Injection on Voice-Controllable Systems	18
Interpretable Deep Learning Under Fire	18
Hiding Objects from Computer Vision by Exploiting Correlation Biases	19
Disrupting Continuity of Apple's Wireless Ecosystem Security: New Tracking, DoS, and MitM Attacks on iOS and macOS Through Bluetooth Low Energy, AWDL and Wi-Fi	20
Defence	21
Entangled Watermarks as a Defence Against Model Extraction	22
Hopper: Modelling and Detecting Lateral Movement	23
Faking a Factory: Creating and Operating a Realistic Honeygot	24
Do You Speak My Language? Making Static Analysis Engines Understand Each Other	25
Practical Defences Against Adversarial Machine Learning	26
Nifty Sundries	27
Remote Side-Channel Attacks on Anonymous Transactions	28
An Observational Investigation of Reverse Engineers' Processes	29
On the Feasibility of Automating Stock Market Manipulation	30
IoT Skimmer: Energy Market Manipulation through High-Wattage IoT Botnets	30
The Dark Age of Memory Corruption Mitigations in the Spectre Era	31
Everything Old is New Again: Binary Security of WebAssembly	31
ProxyLogon is Just the Tip of the Iceberg: A New Attack Surface on Microsoft Exchange Server!	32
Conclusion	33



Introduction

*Like many parts of the world now emerging from an undesired hiatus, ThinkstScapes is returning afresh to highlight research that is interesting, unexpected, or opening the door to new areas of exploration. Having previously been a private, **paid-for publication**, ThinkstScapes will now be available to everyone for free. We aim to help everyone sort signals from the research noise.*



Hopefully this edition will help call attention to some of the novel and impactful research projects published during the pandemic.

If this is the first time you are reading an issue of ThinkstScapes, this is a quarterly review of information security research published in both industry and academic venues. The deluge of research content produced by security practitioners and researchers means that it's almost impossible to stay up to date with interesting research. Worse, without devoting a significant amount of time tracking them, it is hard to identify useful trends and themes. We aim to help by highlighting interesting research, specifically looking for novel and unusual work that is impactful – this means not simply finding a report on bugs or vulnerabilities (although they will crop up in the context of interesting work).

Work covered here will include both offensive and defensive topics, and we explore academic publications with the same gusto as industry work. Although our target readers are primarily security practitioners in organisations where they are tasked with defending their turf, offensive-minded folks will also be exposed to new ideas and research we've come across.

The last ThinkstScapes issue was published in 2016, and the world today is a very different place. For this comeback edition, we've settled on an intermediate coverage period that is longer than a quarter but doesn't extend back the full five years. We're going to cover what's happened during a *little* thing called COVID-19.

It is hard to avoid the cliché of mentioning how drastically the world has changed due to the pandemic, but it has clearly had a profound impact on information security research too.

This can be felt in the topics tackled, how research was performed and the way it was presented. If you are anything like us, the ability to watch talks without leaving the house quickly grew old as the screen time committed to video calls grew unabated. Hopefully this edition will help call attention to some of the novel and impactful research projects published during the pandemic.

In addition to the move to virtual presentations, other changes have emerged in information security since the release of the last ThinkstScapes in 2016. Five years is a long time in our world, and in this time the industry has seen significant shifts. In particular, the rise of speculative execution vulnerabilities and attacks has had a major impact on shared services (i.e., cloud computing), and has undone decades of microarchitecture optimisations. An entire new industry in the form of cryptocurrency security has taken root, with rampant attacks seen and specialist skills required to audit and test smart contracts. Machine learning continues to rise in practical use, with both attacks and defensive work following. Containers and container orchestration have elbowed their way into production networks around the world, necessitating hard examinations of these new attack surfaces.

The dominance of client-side exploitation has faded as sustained efforts on the part of browser and operating system vendors drive up the difficulty (and hence the cost) of compromise. This has led to renewed interest in server-side vulnerabilities and attacks. New web-hacking techniques, once all the rage, are now relatively infrequent. And the cloudification of core services like authentication, mail and identity management (once considered unthinkable) is now commonplace, leading to whole new classes of vulnerabilities such as Golden SAML, malicious O365 applications, and more.

In terms of numbers, in 2019, leading up to the pandemic, Thinkst tracked 6,940 talks catalogued across over 1,700 conferences. 2020 had 5,313 talks across slightly more than 500 conferences (a majority of them in virtual format).

● In 2019

Thinkst tracked **6,940 talks**
catalogued across
over **1,700 conferences**

● In 2020

Thinkst tracked **5,313 talks**
across slightly more than
500 conferences
(a majority of them in virtual format)

THIS EDITION INCLUDES TALKS DRAWN FROM THE FOLLOWING CONFERENCES:

(listed with the number of presentations)

Conference	Number of talks
ACSAC 2020	70
CanSecWest 2021	18
RSAC 2021	265
IEEE S&P LangSec 2021	16
IEEE S&P LangSec 2020	12
USENIX Security 2020	157
USENIX Security 2021	246
Black Hat USA 2020	94
Black Hat USA 2021	95
Black Hat EU 2020	39
Black Hat Asia 2020	40
Black Hat Asia 2021	40

Themes covered in this issue

INTO THE EMBEDDED REALM

Security research into embedded devices or other low-level system targets has certainly had its ups and downs, from being dismissed as “junk hacking” to the spate of recent micro-architectural/speculative execution attacks. Talks in this section demonstrate the increased impact of embedded/low-level security research: no longer are the vulnerable devices just cheap IoT endpoints, they now are embedded in the computers, mobile devices and the critical infrastructure that we depend upon. This theme includes many open-sourced tools which allow for any researcher to gain the visibility that the authors had in order to continue their digging into these oft-forgotten niches. Each work can show the potential for significant impact from their discoveries (including a number of remote attacks on embedded devices), and it is safe to say that these papers will not be as easy to dismiss.

DEFENCE

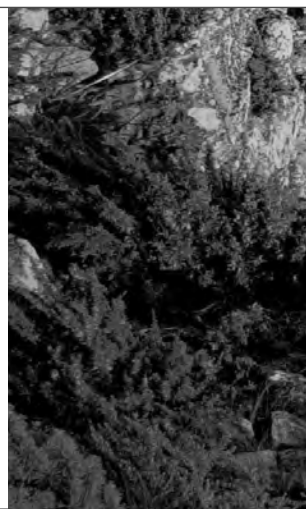
Defensive research is hard to evaluate in a repeatable or meaningful way – did it take the red team longer to move through the network due to the defensive improvements, or were they simply having an off day? Because attackers can be dynamic, adaptive and creative, it is difficult in defence to find the intersection of a realistic threat model, empirical evaluation and high-impact outcomes. Talks in this section cover research from real-world defences against in-the-wild ML attacks versus the theoretical back-and-forth in academic literature, to scaling static analysis across repositories and languages, to the results from building an entire fake factory and corporate entity as a high interaction honeypot.

NIFTY SUNDRIES

With so many great talks and papers to choose from, there were a number that we felt should be highlighted but didn't directly fit into any of the aforementioned themes. Please enjoy reading about everything from attacks on privacy coins, to user experience studies of reverse engineering tools, to good ol' fashioned exploitation of email servers (with a twist).

EXPLOITING “DIFFERENCES OF OPINION”

At every bug's core lies some broken assumption or misinterpretation. The works highlighted in this section show how broadly these differentials in understanding can impact security. From HTTP/2 downgrades re-allowing de-sync attacks, to cross-physical-modality pivots that trigger unexpected results, these papers show how the complexity of systems in-common-use expands the number of interfaces exponentially where these confusions can occur.



Into the embedded realm

- ✓ *Precursor: Towards Evidence-Based Trust in Hardware*
- ✓ *Kernel Pwning with eBPF: a Love Story*
- ✓ *InternalBlue / Frankenstein / Spectra*
- ✓ *HALucinator: Firmware Re-hosting Through Abstraction Layer Emulation*
- ✓ *Device-agnostic Firmware Execution is Possible: A Concolic Execution Approach for Peripheral Emulation*
 - ✓ *Remote Timing Attacks on TPMs, AKA TPM-Fail*
 - ✓ *Breaking VSM by Attacking SecureKernel*
 - ✓ *Whispers Among the Stars: Perpetrating (and Preventing) Satellite Eavesdropping Attacks*



PRECURSOR: TOWARDS EVIDENCE-BASED TRUST IN HARDWARE

Watch
Video

Author: Andrew 'bunnie' Huang

This keynote on the Precursor hardware project highlights an unaddressed issue: most people have no way to verify that the hardware they've purchased is trustworthy. It is a general assumption that the laptop, phone, router, or Google Home that arrives via courier hasn't been opened and modified while en route. We also take as a ground truth that none of the components were swapped out for counterfeit or malicious replacements on their way to the assembly line. Only the most paranoid would even pause to consider that the PCBs printed in faraway factories could have had their designs surreptitiously modified after the engineers signed off on them, to say nothing of the firmware loaded afterwards.

There is no way (yet) to check that your phone is exactly what the designers intended you to get; even if you could somehow check the hardware, verifying firmware is next to impossible without specific assistance in the designs. The closed ecosystems we've drifted towards explicitly don't expose such functionality.

This is an immense challenge to tackle, and this work is taking steps to explore and solve parts of the problem space. This nascent field of verifiable hardware has three principles the author identifies as necessary for "evidence-based trust in hardware": complexity must be avoided (e.g., devices should be inspectable without destructive disassembly), the entire system must be verified (as opposed to individual components since security issues often occur at boundary interfaces)

and end-users should be able to verify and seal their hardware.

Precursor is a prototype research device designed and built as an example of how evidence-based trust can be brought to hardware supply. End-users can verify the design, it has simple components without built-in processors, and the problem of trying to prove an unaltered SoC is avoided by shipping an FPGA instead. Users can load their own CPU implementation onto the FPGA.

Precursor is very much a demonstration platform. While it has some uses, it's a low-powered device. It also doesn't completely remove the physical TOCTOU vulnerabilities, but it does drastically reduce them. The author is vocal about these shortcomings, and his desire to tackle one of the hardest problems in our industry is refreshing.

TAKEAWAYS:

- ✓ Hardware supply chain issues usually get some kind of policy response. But there are technology possibilities here that are worth exploring.
- ✓ It's not clear that this is scalable to higher-performance machines, or if this will be relegated to low-powered devices. If so, will the impact be limited if only applicable to low-cost devices that are too numerous to manually audit?

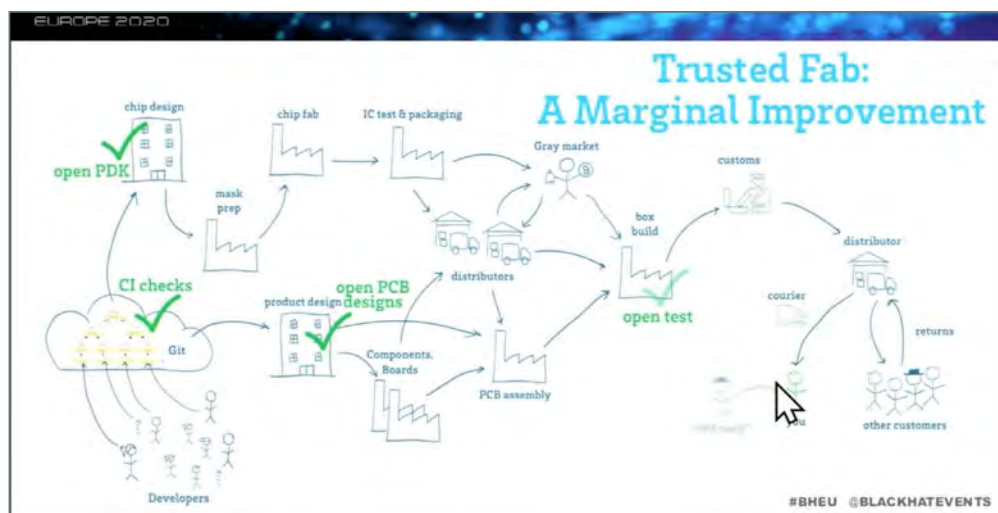


Figure 1: Physical TOCTOU vulnerabilities exist throughout hardware supply chains.

KERNEL PWINING WITH EBPF: A LOVE STORY

[Read Paper](#)

Author: Valentina Palmiotti • @chompie1337

It is unusual to include a standalone bug or exploit write-up in an issue of ThinkstScapes. Despite that, this work has been included because eBPF's star is currently on the rise (and because of the quality of the researcher's write-up).

In a detailed post on their blog, the author gives a quick overview of eBPF (and its verifier) and then proceeds to explain her thinking while attempting to craft an exploit for a [local privilege escalation bug](#) revealed three months prior.

eBPF is an increasingly popular way to extend the capabilities of the Linux kernel while promising a safe, sandboxed environment. It enjoys increasingly widespread adoption and just this week saw the running of the second annual [eBPF summit](#) with speakers from AWS, Microsoft, Netflix and other industry heavy hitters.

A fair bit of work has gone into comforting people that eBPF would be resistant to abuse. A great deal of this is handled by the verifier, which was targeted for abuse in this instance. By exploiting a logic bug in the verifier,

the author was able to effect a DoS, an information leak, and ultimately a local privilege escalation. In addition to the unusually clear bug write-up, the author also lays out useful steps for debugging eBPF-related exploits going forward.

This write-up follows hot on the heels of another interesting [eBPF exploit released by the Qualys team](#) just a few weeks earlier. Brad Spengler (once again) had made [hardening suggestions](#) for eBPF back in 2016, which are starting to look more and more reasonable.

TAKEAWAYS:

- ✓ Although eBPF offers a bunch of new possibilities for observability and networking, it also introduces a new possible attack surface.
- ✓ The current state of the eBPF verifier doesn't instill confidence in cases where the submitter of the eBPF bytecode is untrusted or semi-trusted.
- ✓ This is likely to be a ripe area for exploitation in the next few quarters.

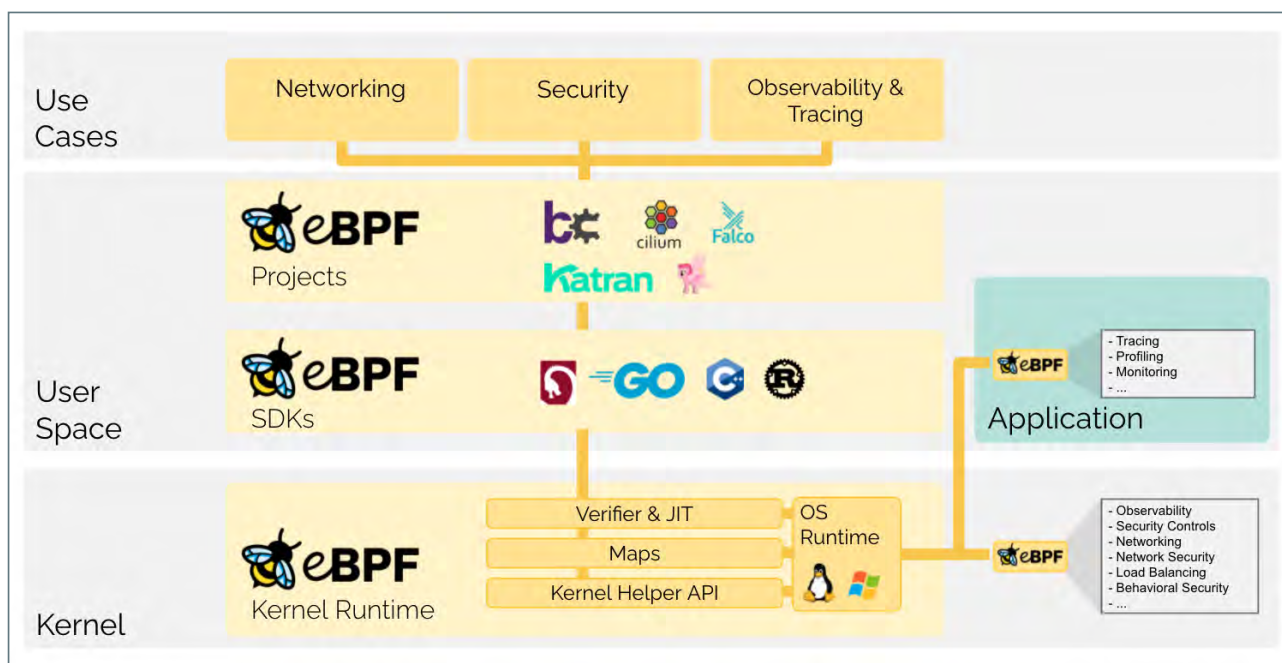


Figure 2: A diagram of the eBPF components.



[View
Slides](#)

[Read
Paper](#)

[Watch
Video](#)

[Read
Paper](#)

APPROACH 1:

HALucinator: Firmware Re-hosting Through Abstraction Layer Emulation

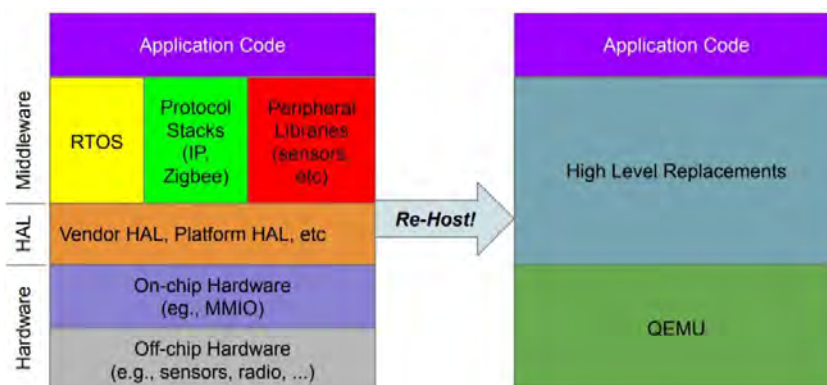
Authors: Abraham Clements, Eric Gustafson, Tobias Scharnowski, Paul Grosen, David Fritz, Christopher Kruegel, Giovanni Vigna, Saurabh Bagchi, and Mathias Payer

APPROACH 2:

Device-agnostic Firmware Execution is Possible: A Concolic Execution Approach for Peripheral Emulation

Authors: Chen Cao, Le Guan, Jiang Ming, and Peng Liu

Figure 4: A figure showing the goal of HALucinator to allow embedded applications to be automatically emulated without building a custom QEMU board support package. The first paper (see Figure 3 above) uses the hardware abstraction layers (HAL) provided in the various chip manufacturers' SDKs as a software stand-in for device peripherals, and automates the mapping of calls to the HAL into the appropriate logic in QEMU.



These two papers take slightly different approaches to solve the same problem: how to take an arbitrary “raw” firmware image for an embedded device and boot the image with sufficient fidelity to perform dynamic analysis (e.g., fuzzing). This re-hosting effort is usually manual and time-consuming, but when complete allows the researcher to drastically scale up their analysis beyond the hardware they have, and the instrumentation limitations of the device. The first paper (see Figure 3 above) uses the hardware abstraction layers (HAL) provided in the various chip manufacturers' SDKs as a software stand-in for device peripherals, and automates the mapping of calls to the HAL into the appropriate logic in QEMU. This was sufficient to fuzz and find novel vulnerabilities in an embedded firmware image.

The second paper takes an even more black-box approach for firmwares that may not have SDKs or public HALs available (e.g., a printer firmware). This paper uses concolic execution (a combination of concrete and symbolic execution) to determine the correct response from the hardware that allows the firmware to continue to execute. As an example, if there is logic in the firmware that polls a memory-mapped IO bit for a peripheral device to wait for it to be ready, this work will determine automatically what

value would allow for the firmware to progress beyond the polling loop. Whilst primarily focused on a specific ARM CPU class, their tool was able to successfully emulate the boot process for 20 of the 30 sample firmware images without any human interaction needed. Once booted, the emulation was sufficiently capable for a fuzzer to find previously-discovered vulnerabilities.

TAKEAWAYS:

- ✓ These tools allow for automation and scaling of vulnerability research on embedded firmwares, a class of software with traditionally poor security that benefited from the manual effort needed up-front to instrument and analyse. As these tools become mainstream, expect growth in the number of embedded vulnerabilities discovered.
- ✓ While these tools make the job of a vulnerability researcher easier, there are still hurdles to overcome in how to deploy patches (or even inform customers) on complex, multi-party embedded supply chains. Without a drastic improvement in firmware security, embedded devices are likely to be plagued with forever-days.



REMOTE TIMING ATTACKS ON TPMS, AKA TPM-FAIL

[View](#)
[Slides](#)

Author: Daniel Moghimi

Timing attacks are well-known as a type of side-channel attack across a range of security mechanisms. In this work, Moghimi looks at the susceptibility of Trusted Platform Modules (TPMs) to timing attacks. TPMs are one of the fundamental components in ensuring a trusted boot process, but their usage extends well beyond that, into security co-processing (e.g., cryptographic operations without the ability to retrieve key material, or hardware random number generation).

The author attempted timing attacks against multiple TPM vendors, including Intel, STMicroelectronics, Nuvoton and Infineon. They discovered that the Intel and STMicroelectronics TPMs were susceptible to local timing attacks, as their actions were not constant time but were influenced by input data. From this it was possible to extract the private keys stored on the TPM, which should never be possible. These local recoveries mostly required system-level access as opposed to user-level, and at most took 80 minutes.

They then showed that the attacks were possible over the network too. By configuring a StrongSwan VPN to rely on the TPM for certain cryptographic operations during session initialisation, they were able to remotely

recover the TPM's key in about five hours (44k handshakes).

Lastly, they described a method of fuzzing libraries for potential timing-attacks. By tracing execution in a cryptographic library with a bunch of random inputs and comparing the execution traces automatically, they were able to identify numerous input-dependent timing leaks in two common closed source libraries (Microsoft CNF and Intel IPP).

TAKEAWAYS:

- ✓ TPMs provide a segregation of security duties, but that doesn't make them magically safe. Previous work has shown issues that can affect TPMs, and in this presentation we see the impact of timing attacks.
- ✓ Significantly the timing attacks are reachable via the network. While the real-world impact is limited as issues were not discovered on all examined TPMs, it's a timely reminder that the network of processors that make up modern computing systems are all likely to be targets.

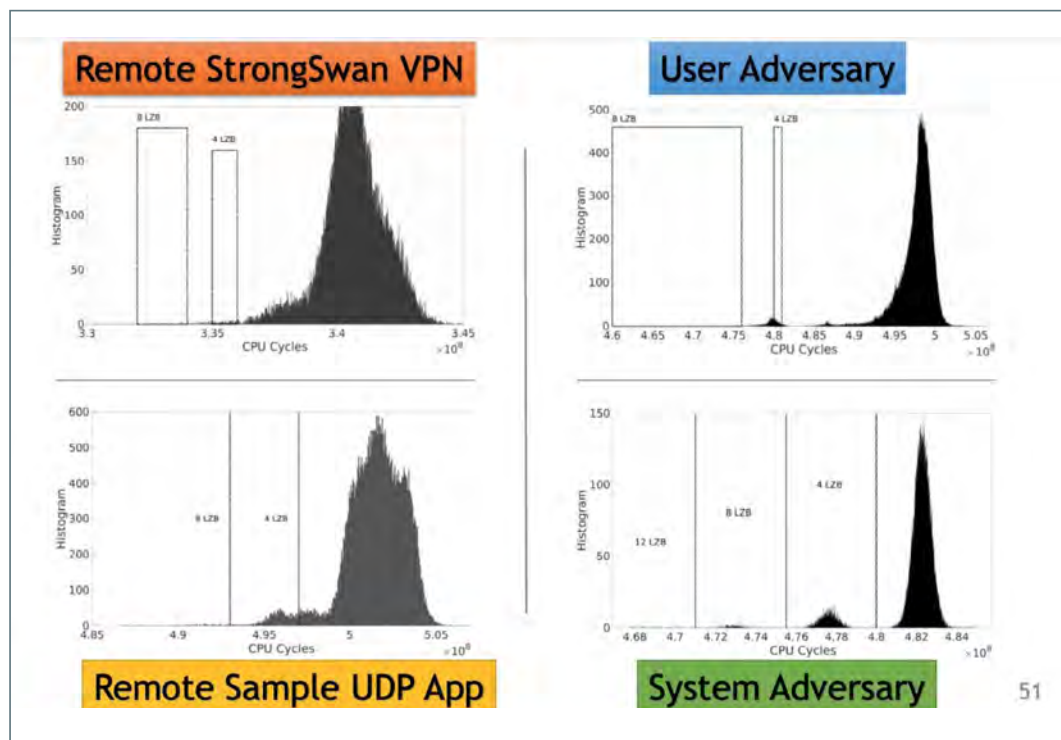


Figure 5: Non-uniform timing distributions across four scenarios.

BREAKING VSM BY ATTACKING SECUREKERNEL

[View
Slides](#)

Authors: Saar Amar and Daniel King

Windows 10 and Windows Server 2016 introduced an additional environment known as the Virtual Secure Mode (VSM), hosted in a separate VM on a Hyper-V base. The purpose of this new virtualisation-based security is to handle security-critical tasks such as cryptographic functions, credential storage, and more. The secure environment consists of its own kernel and user-space, separate from the standard Windows environment consisting of the `ntoskrnl.exe` kernel and the Windows userland.

By moving those functions into a separate environment with access limited via the hypervisor, tasks running in the secure environment have much greater protection than when they share the same environment as the rest of the OS. For example, malware running in the normal environment cannot simply scrape the memory of sensitive processes running in the secure environment, even if the user running the malware has administrative privileges.

To this end, VSM marked a departure from the normal Windows architecture. Of course, with a new kernel and additional functionality to segregate functionality between VMs comes the opportunity for new problems. There has been very little in the way of public research into the VSM exploitation, and

this work from folks at Microsoft Security Response Center is a welcome addition. The researchers show the interface between the normal and secure environments, and describe how they fuzzed it, discovering bugs in this interface. They then described useful exploit primitives before showing an exploit that gained arbitrary code execution in the secure environment from the normal environment. They went on to find an additional vulnerability along with a second exploit.

Solely finding and exploiting bugs isn't normally grounds for inclusion in ThinkstScapes, but in this case the novel work was exposing the internals of VSM and providing insight into designing exploits that cross this virtualisation-based boundary.

TAKEAWAYS:

- ✓ The VSM included in Windows from Windows 10 and Windows Server 2016 onwards carries an additional environment that is also subject to bugs. This work provides direction for future research into VSM.
- ✓ As always, security issues congregate (or, at least, are reachable) at the interfaces.

Figure 6: An outline of the overall exploit chain for VSM.

Exploit #1 – Final to Arbitrary Code Execution

- Corrupt MDL->Next, gain 1 arbitrary write
- Fake a pivot MDL structure in the shared page (simply writes in VTLO)
 - Keep in mind that we can change that repeatedly, by design
- Use the arbitrary write to corrupt a node in `SkpLiveDumpContext.Mdl` chain, make it point to our pivot MDL
- Call `SkLiveDumpAddBuffer` to trigger arbitrary write
- Change shared page content, and call `SkLiveDumpAddBuffer` again!
- Arbitrary Write: Corrupt PTE --> make shared page RWX
- Arbitrary Write: Corrupt `SkpgContext` callback --> jump to shellcode
- PROFIT



WHISPERS AMONG THE STARS: PERPETRATING (AND PREVENTING) SATELLITE EAVESDROPPING ATTACKS

[View
Slides](#)

[Watch
Video](#)

Author: James Pavur

Continuing a long trend of research on the weaknesses of commercial (and at times military) satellite networks, this talk demonstrated the type of information visible to an adversary with ~\$350 of equipment. Due to the latency and cost of GEO-based constellation use, end-to-end encryption (such as VPNs) is rarely deployed, despite sensitive information being transmitted between ground and overhead assets. The author built tools to attempt to recover streams from data captured via a small dish, successfully recovering cleartext streams from IoT, maritime and aviation endpoints. These streams contained not only unencrypted personal communications (e.g., ship-board emails and SMS messages from in-flight portals) but also updates to maritime charts and aircraft data. Finally, some possible active attacks are presented in which a TCP session can be hijacked using the TCP sequence numbers to inject traffic to the destination.

TAKEAWAYS:

- ✓ Expensive, high-latency network links add additional challenges to deploying end-to-end encryption for OT systems. This, in addition to rarely (if ever) patched implementations of legacy crypto (when present at all), means there is a long tail of insecurity for critical infrastructure.
- ✓ The deployment of faster and cheaper LEO-based internet constellations should allow those organisations that can move from the legacy GEO networks to improve not only speed, but also security when the network is more similar to traditional wired ISPs.
- ✓ MitM attacks at ISP-level rarely present themselves to non-nation state actors. Therefore trusting the connection from ISP to destination may not always be safe, even against less powerful adversaries.



... end-to-end encryption (such as VPNs) is rarely deployed, despite sensitive information being transmitted between ground and overhead assets.

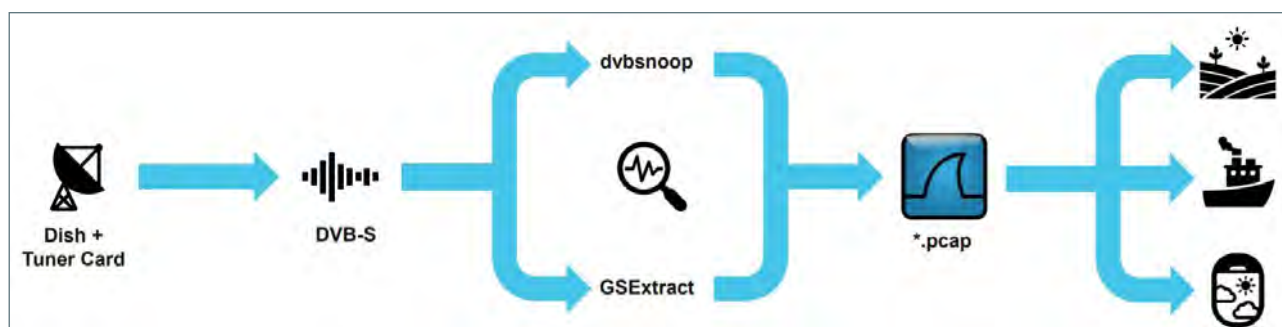


Figure 7: A depiction of the data recovery flow from a \$350 receiver unit to cleartext network traffic captures.

Exploiting 'Differences of opinion'

- ✓ *HTTP/2: The Sequel is Always Worse*
- ✓ *Differential Analysis of x86-64 Instruction Decoders*
- ✓ *EtherOops: Exploring Practical Methods to Exploit Ethernet Packet-in-Packet Attacks*
- ✓ *Light Commands: Laser-Based Audio Injection on Voice-Controllable Systems*
- ✓ *Interpretable Deep Learning Under Fire*
- ✓ *Hiding Objects from Computer Vision by Exploiting Correlation Biases*
- ✓ *Disrupting Continuity of Apple's Wireless Ecosystem Security: New Tracking, DoS, and MitM Attacks on iOS and macOS Through Bluetooth Low Energy, AWDL and Wi-Fi*

HTTP/2: THE SEQUEL IS ALWAYS WORSE

[Read
Paper](#)

Author: James Kettle

New classes of web attacks aren't discovered as frequently as they used to. A decade (or more) ago, the world of web security was a flurry of activity, with novel attacks being the subject of mere [!] blog posts. But this curve has flattened, and while web attacks continue unabated, truly new attack vectors are seldom seen.

This author has been carrying the banner for new types of web attacks over the past few years. He reimagined request smuggling in modern sites, tackled cache attacks last year, and now has taken aim at HTTP/2. The goal of this work was to repeat the [HTTP desync attacks](#) in HTTP/2, and along the way he discovered an additional morsel in the form of request tunnelling. HTTP/2 is a binary version of the well-loved HTTP/1.1 protocol, and brings along a bunch of performance improvements; it currently accounts for slightly less than half of all web traffic.

The author points out that, in a typical HTTP/2 deployment, only the connection between the client and front-end proxy actually uses HTTP/2. Often the communications from the front-end proxy to internal systems are then downgraded to HTTP/1.1. This downgrading is what's at the heart of the issue. The front-end web server must translate between HTTP/1.1 and HTTP/2, and there's lots of scope for mistakes to creep in.

```
:method POST
:path /identify/XUI
:authority id.b2b.oath.com
transfer-encoding chunked
0

GET /oops HTTP/1.1
Host: psres.net
Content-Length: 10
x=
```

Figure 8: Combining a “banned” transfer-encoding header with an embedded request.

For example, HTTP/2 doesn't require a Content-Length header since the binary protocol already allows for length fields, but the standard doesn't prevent a client from including one. Certain sites would pass along an unverified Content-Length header from the HTTP/2 front-end to the downstream application server in an HTTP/1.1 connection, which would get confused about where the request actually started.

Similarly, it's possible to include all sorts of characters in request lines and paths, duplicate fields, embed requests-in-requests, and more.

If HTTP/2 was maintained end-to-end (i.e., HTTP/2 between client and front-end, and between front-end and application server), this issue wouldn't exist in its current guise. But the (numerous) bounties collected as part of this work showed that downgrading appears common.

TAKEAWAYS:

- ✓ Web attacks have slowed in their novelty, although not at all in their prevalence.
- ✓ Issues (as always) appear in the cracks and in the interfaces. The HTTP/2 to HTTP/1.1 downgrade process is ambiguous enough that many inconsistencies were discovered in multiple implementations.

DIFFERENTIAL ANALYSIS OF X86-64 INSTRUCTION DECODERS

[Read
Paper](#)

[Watch
Video](#)

Authors: William Woodruff, Niki Carroll, and Sebastiaan Peters

Astutely identifying the growth in deployed binary analysis and run-time translation (e.g. Apple's Rosetta translator to support x86 applications on the M1 ARM Macs), this work targets these components with a differential fuzzing framework to search for instructions that are decoded differently by different decoders. Differential fuzzing works not by searching a single target for a crash (or other unexpected behaviour), but by providing the same input to

multiple targets that should process that input in the same manner (and then looking for deviations). Past work in automatic discovery of instructions which are incorrectly decoded aimed to look for semantic deviance between a software decoder and a “golden” decoder, such as the hardware decoder in the CPU. This work instead cuts out the golden decoder, and cohorts decoders looking for those that differ from the majority consensus.

By removing the hardware instrumentation overhead, the decodes per second can be increased, and greater coverage of the instruction set can therefore be obtained. The paper presents the initial results from a four-hour fuzzing run (a minuscule amount of time by modern standards), discovering tens of millions of instructions that were either deemed incorrectly valid or invalid by one or more decoders, or decoded to different semantics between the cohort. While the results of this paper only point to the impact stemming from these underlying issues, future work includes building an automatic generator for executables which behave differently depending on the decoder (e.g., an application that appears benign to an AV analysis engine but executes malicious code when run natively).



Figure 9: Flow-chart of differential fuzzing.

TAKEAWAYS:

- ✓ As computer performance improves, the cost calculus for porting code from one platform to another versus translating/emulating is changing. Binary analysis libraries, once used primarily for legacy emulation and security research, now underpin major operating systems (Rosetta) and web applications (WebAssembly).
- ✓ Traditional fuzzing relies on the notion that every application has an informal guarantee that it should not crash on arbitrary input – violating that means a bug.

Differential fuzzing allows for more nuanced crafting of inputs that allow for the same input to be understood differently by different parsers. Perimeters reliant on WAFs, AVs and other analysis products become more permeable if you can tune the input to bypass one engine and strike at another.

ETHEROOPS: EXPLORING PRACTICAL METHODS TO EXPLOIT ETHERNET PACKET-IN-PACKET ATTACKS

[View
Slides](#)

[Read
Paper](#)

Authors: Ben Seri, Gregory Vichnepolsky, and Yevgeny Yusepovsky

This research took another look into the feasibility of a packet-in-packet (PiP) attack against a wired Ethernet network. PiP was originally described as an attack against certain 802.11 and 802.15 wireless protocols (Wi-Fi and Zigbee, respectively), where an error in the lower levels of the OSI stack would allow for the misinterpretation of the packet payload as a raw PHY frame. Earlier works had examined the possibility of PiP attacks against wired networks, where the error rates are significantly lower, and concluded that it is unrealistic. The authors re-examine this attack by looking at a large dataset. They find a small percentage of deployed cables exhibit higher-than-specified errors, allowing for a successful attack (injection of a single frame behind a firewall) on the order of hours. Various attack scenarios are presented, including an IPv6 router announcement that directs clients to update their DNS and proxy detection to an attacker-controlled value. Finally, experiments are performed on using EMP attacks to artificially increase the error rate to decrease attack time.

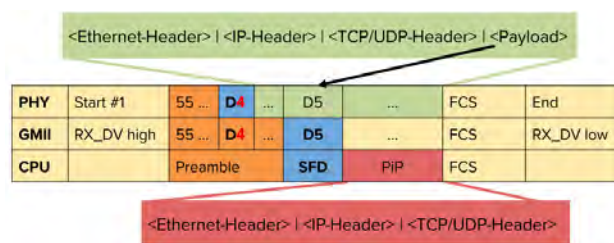


Figure 10: Diagram showing the payload misinterpreted as a raw Ethernet frame.

TAKEAWAYS:

- ✓ Whilst it is still an unlikely attack, faulty Ethernet cables are rarely considered enablers for remote attacks. The impact of this is amplified when OT and legacy devices will rely on perimeter defences for the foreseeable future.
- ✓ The interconnected (and defaultly enabled) IPv4 and IPv6 stacks add hidden complexity and allow for malicious effects, even if the network is not designed to support dual stacks.



LIGHT COMMANDS: LASER-BASED AUDIO INJECTION ON VOICE-CONTROLLABLE SYSTEMS

[View Slides](#)

Authors: Takeshi Sugawara, Benjamin Cyr, Sara Rampazzi, Daniel Genkin, and Kevin Fu

This research is fun in a Mission Impossible kind of way, and it's easy to explain the concept. It turns out that the MEMS microphones used in a bunch of Voice Controllable Systems (VCS) such as the Amazon Alexa range, Google Home devices, certain mobile phones, and more, can be triggered via Lasers across distances up to 110 meters. In other words, by converting an audio signal such as "OK Google, Open the garage door" into a series of Laser pulses, the microphone is activated through two different physical effects, one photoelectric and the other photoacoustic. The signal fed by the MEMS microphone into the VCS maps to the same audio signal, and so it means "speaking" into the VCS through transparent barriers and at a significant distance.

The real-world implications are somewhat constrained by the fact that VCSs aren't typically found in sensitive environments, and any attack would have to be extremely targeted (you might even say, *laser-focused*!). That said, crossing physical modalities is an interesting pivot, and may just

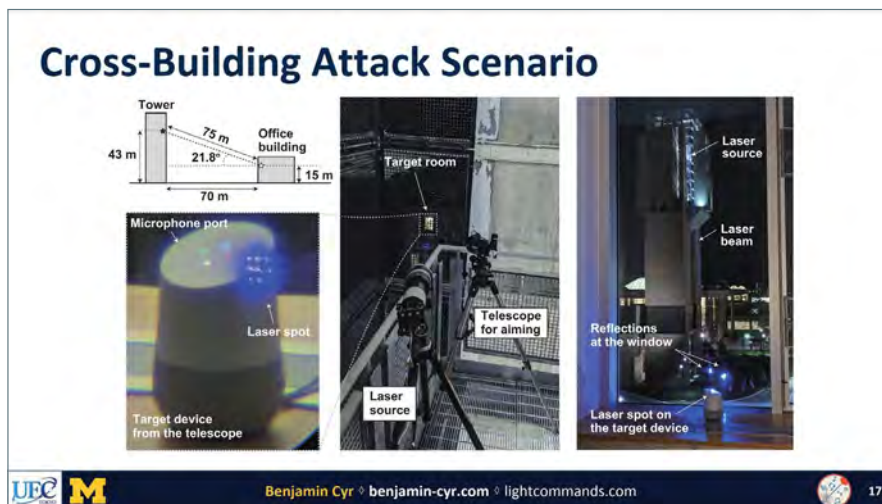


Figure 11: Attack setup to remotely inject audio from afar.

persuade you to move your Echo or Google Home away from the windows.

TAKEAWAYS:

- ✓ Physical attacks can sometimes look like Hollywood stunts.
- ✓ This is a reminder that in the physical world, neat isolation is seldom achievable and an unexpected physical property can affect the security of any system.

INTERPRETABLE DEEP LEARNING UNDER FIRE

[View Slides](#)

[Read Paper](#)

[Watch Video](#)

Authors: Xinyang Zhang, Ningfei Wang, Hua Shen, Shouling Ji, Xiapu Luo, and Ting Wang

Interpretable machine learning, or explainable AI, is a new advancement in building trust in the decisions made by a machine learning model by providing transparency into why a decision or classification was made. These explanations, when paired with a human monitor, provide the ability to teams where the human can observe the model and understand which features (or region of input) lead the model to choose a specific output class. The techniques for generating explanations include highlighting the regions of input that the model used, but can also break down a decision into various rationales. For example, the

object in a picture is classified as a "cat" because it has whiskers and pointy ears. To generate these explanations from a high-dimensional model, generally another model is used – this research highlights the fact that there is a gap between the classification model and the interpretation model. By training an adversarial input generator to optimise for both of these (i.e., minimise perturbation of image while maximising change in output class and minimising change in interpretation), adversarial samples can be generated that fool the classifier while providing a reasonable explanation.

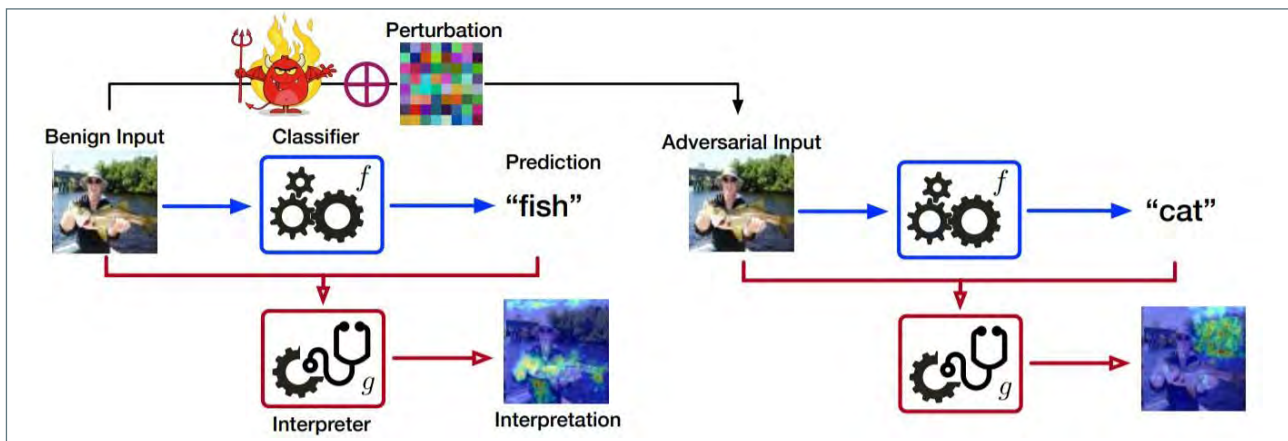


Figure 12: A figure showing how the explainability or interpretation layer can act as a secondary check (g) on the classification output (f) of a ML model.

TAKEAWAYS:

- ✓ Just as the classification model acts as a lossy compression function from input space to output labels, so too is there lossiness in the mapping from a classification to the interpretation. As long as there is a semantic gap between these spaces, exploitation will be possible.
- ✓ Nowhere is the game of cat-and-mouse so clear as in adversarial ML. Seemingly each defensive strategy is incorporated as an additional constraint to optimise against and retrain; each attack is used to generate an adversarial training corpus to thwart that class of attack.

HIDING OBJECTS FROM COMPUTER VISION BY EXPLOITING CORRELATION BIASES

[View
Slides](#)

Authors: Yin Minn Pa Pa, Paul Ziegler, and Masaki Kamizono

This research took a novel attack angle on the correlation bias most notably demonstrated in a [2016 paper](#) by Ribeiro et al. The 2016 work showed how classifiers trained to differentiate wolves from dogs overwhelmingly base their decision on the presence of snow in the background – wolves are more often photographed in the snow when compared to dogs. This correlation bias has been used both as a data-point in the fairness of AI/ML algorithms, and as a demonstration of the lack of context in traditional ML models and how that can be a weakness. To drive that demonstration point home, this work shows how combining objects rarely seen in the same image together during training prevents detection or classification.

As an example, the stop sign image was detected with high confidence when seen as part of a roadway scene. When the detected region was overlaid on a bowl of fruit, however, the computer vision algorithms rarely detected that there was a stop sign in the image. Instead of the typical attacks where minute changes are introduced to the noise in the image, this work attacks the correlation of detected objects, and was

replicated for physical scenes (e.g. paying for a billboard of fruit that is behind a traffic sign and having autonomous vehicles ignore stop signs). Fortunately, only a minor number of adversarial examples are needed when training a ML model to mitigate these attacks.



Figure 13: Composite image with stop sign, which is easily detected in an image of a road. When combined with a bowl of fruit, however, it is no longer detected.

TAKEAWAYS:

- ✓ Machine learning is increasingly embedded into the products that surround us, and remains an opaque and unreliable process that can fail with high-certainty in specific edge cases. This work highlights yet another failure mode stemming from both the lack of actual intelligence and the opacity of these high-dimensional statistical models.



DISRUPTING CONTINUITY OF APPLE'S WIRELESS ECOSYSTEM SECURITY:

New Tracking, DoS, and MitM Attacks on iOS and macOS Through Bluetooth Low Energy, AWDL and Wi-Fi

Read
Paper

Authors: Milan Stute, Alexander Heinrich, Jannik Lorenz, and Matthias Hollick

This research explored the integrations across Apple's mobile ecosystem – with over 1.5 billion active devices, any built-in service that runs on them is an interesting target for attackers. Specifically, the authors examined Continuity, a set of 12 different wireless services that run on Apple devices using Apple's custom and proprietary protocols. Whilst there has been a wide range of attacks against standard wireless protocols and implementations, very few of the Continuity services have been publicly explored or attacked.

Although the authors did report several security and privacy vulnerabilities to Apple during the course of this work, we are more interested in their work dissecting and documenting the protocols. They released tooling under the Open Wireless Link ([OWL](#)) project that includes Wireshark dissectors, and open source implementations of AWDL and AirDrop.

Their paper also includes a kind of “101 guide” for how to approach closed source binary protocol implementations on a limited budget. The walk-through alone is worth reading for newcomers to help demystify a process that sometimes seems so out of reach, while actually containing a series of fairly mundane steps.

The authors have also included links to several GitHub repositories, including their Python-based [“Apple Continuity Reverse Engineering Toolkit”](#).

TAKEAWAYS:

- ✓ Apple's Continuity services bundle lots of eggs in a few central baskets and so far have gotten away pretty unscathed from attacks against them.
- ✓ Fundamental work like this, and tool chains to get researchers working on higher levels of the stack usually precede the shaking out of bugs.
- ✓ We think the space is both interesting to attack, and complicated enough to hide interesting problems and will be watching to see where this goes.



Figure 14: Apple's Continuity Services.

Service	AWDL	BLE	Wi-Fi
Handoff (HO)	✓	✓	✓
Universal Clipboard (UC)	✓	✓	✓
Phone	✗	✗	✓
SMS	✗	✗	✓*
Instant Hotspot	✗	✓	✗
Auto Unlock	✓	✓	✗
Continuity Camera	✓	✓	✓
AirDrop	✓	✓	✗
Apple Pay	✗	✓	✗
Wi-Fi Password Sharing (PWS)	✗	✓	✗
Sidecar	✓	?	✗
Continuity Markup and Sketch	✓	?	✗

Figure 15: Table showing the support wireless protocols.

Defence

- ✓ *Entangled Watermarks as a Defence Against Model Extraction*
- ✓ *Hopper: Modelling and Detecting Lateral Movement*
- ✓ *Faking a Factory: Creating and Operating a Realistic Honeypot*
- ✓ *Do You Speak My Language? Making Static Analysis Engines Understand Each Other*
- ✓ *Practical Defenses Against Adversarial Machine Learning*



ENTANGLED WATERMARKS AS A DEFENCE AGAINST MODEL EXTRACTION

Read
Paper

Authors: Hengrui Jia, Christopher A. Choquette-Choo, Varun Chandrasekaran, and Nicolas Papernot

The rise of Machine Learning over the past decade is old news at this point. Our last ThinkstScapes (in 2016) included an introduction to adversarial machine learning. The topic receives much attention, and at this stage there's a metapoint to be made. In the same way that groundbreaking memory corruption talks assume a significant working knowledge of the domain, so too does machine learning research as it pertains to security. The novel work typically shines a light on obscure and unexplored areas, but a significant mental map is assumed on the part of the reader to know the route to the start of exploration.

Machine learning talks are now at this stage of proposing defence, attack, counter-attack, much like memory corruption. They're in an ever-escalating cycle.

Model extraction is a major concern for organisations that have poured significant training resources into their models. In this talk, the authors describe a mechanism to watermark machine learning models. The goal isn't to prevent extraction; rather, it's to be able to assert ownership if extraction is suspected. Older techniques would insert "triggers", or unexpected values, such that when presented with an input holding the "trigger", the model would return a clearly

identifiable response. The issue with these triggers is that common model extraction techniques are unlikely to include the trigger in their inputs to the model, thus the watermark wouldn't carry across into the copied model.

The authors improve on previous watermarking attempts, by "entangling" values from a watermark distribution with the training data. There is no single triggering input that returns a "watermarked" value. Instead, if a model owner suspects that their model has been extracted, they make a number of queries against the stolen data. For certain parameters, the owner can be 95% certain with just 30 queries.

It's also worth highlighting that the authors specifically include a section on adaptive attackers. In other words, trying to think how model extractors may respond to such a defence.

TAKEAWAYS:

- ✓ Machine learning security research is in a cycle of attack, defence, counter-attack. Expect work to be arcane and highly specific to the previous "round" of attacks or defences.

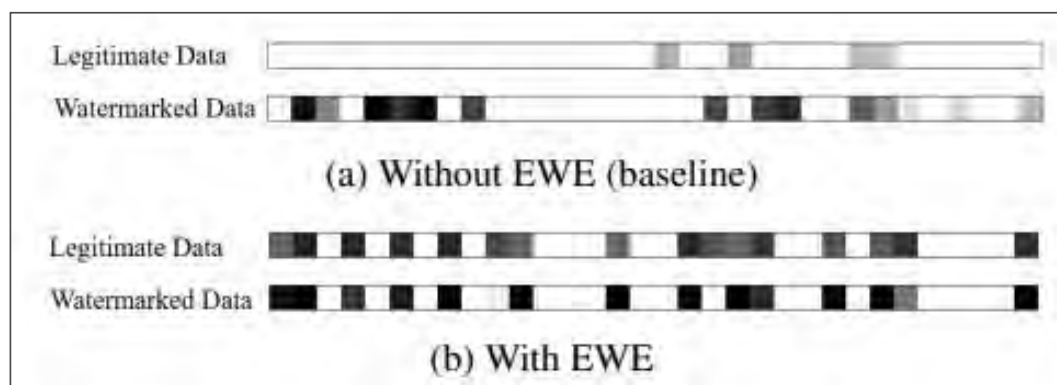


Figure 16: A figure showing how a high-confidence of theft can be obtained from multiple data samples.

HOPPER: MODELLING AND DETECTING LATERAL MOVEMENT

[Read Paper](#)

Authors: Grant Ho, Mayank Dhiman, Devdatta Akhawe, Vern Paxson, Stefan Savage, Geoffrey Voelker, and David Wagner

It is not often that a security-ML paper admits any shortcomings of applying ML to every problem. We are always pleased when we do find an honest ML paper that can demonstrate value and be candid about shortcomings. The Hopper paper does a good job of using ML where it can add value in flagging suspicious lateral movement, while preventing alert fatigue and the traps of blindly using ML as a hammer. The authors use some initial structured data analyses to group logins between multiple systems into either “benign” (a user who initiates a login has access to that resource), or “clearly suspicious” (a user who switches credentials as they move through the network and access a resource the initial user didn’t have permission to.) A final category (“unclear”) is for logins where a number of users login to another system, and from there connect out, making it unclear if there is credential switching or not.

They conduct training on two months of login data across all of DropBox, then look for a red team in another 13 months of data. The authors are able to flag red team behaviour, while limiting false positives to

single digits per day. The constraints and assumptions on the red team were not described in much detail, and a number of clear lateral movement techniques would slip through this system (like using a scheduled task/script/malware to move), but designing the system to include an alert budget to prevent spamming incident response is a great step to reduce SOC fatigue.

TAKEAWAYS:

- ✓ Organisations with fewer legacy components result in cleaner data for training and classification of ML-based systems, furthering the security divide between legacy tails and modern stacks.
- ✓ With a ROC curve showing the false positive rate, alerting can be tuned to minimise fatigue while maximising the probability of correct detection.
- ✓ Alert budgets are an excellent idea and should be part of the conversations when building monitoring.

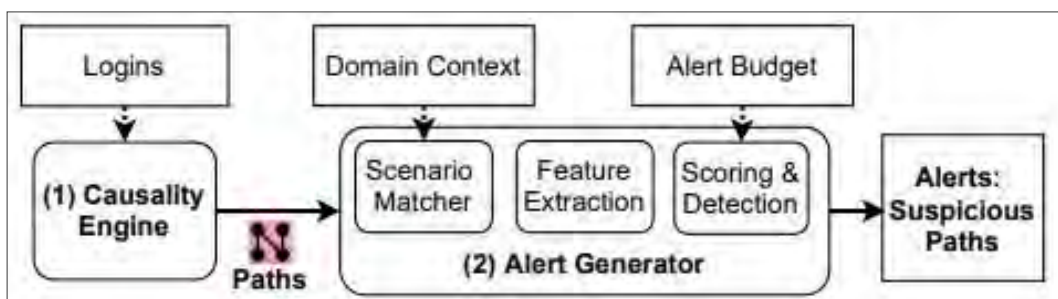


Figure 17: Hopper architecture ingesting Login logs, context about the system and an alert budget to flag suspicious activity.

FAKING A FACTORY: CREATING AND OPERATING A REALISTIC HONEYPOT

[View
Slides](#)

[Read
Paper](#)

[Watch
Video](#)

Author: Charles Perine

Honeypots exist on a spectrum of low- to high-interaction; from an SSH daemon that never allows logins, to VMs setup to monitor how attackers move through a virtual network. This work takes high interaction to the next level, not only building a virtual IT network, but also a full virtual SCADA factory, and fake company website – complete with corporate personas and realistic addresses. Once the factory and its IT network were set up, it was left with some glaring security holes and monitored closely to see how it was attacked. The figure shows a timeline of attacks, almost exclusively on the IT network (two attackers did access the HMI, but did not maliciously try to interrupt the process flow).

In the slides and paper, the author describes many of the attacks, including some notes left both by attackers and an internet-based Good Samaritan. Without the fabricated company serving any critical roles in supply chains, government systems, and so forth, the honeypot was targeted by low-skilled attackers, including a ransomware actor who renamed files instead of encrypting them.

TAKEAWAYS:

- ✓ Even with the high-fidelity “factory” OT environment, none of the multiple attackers who accessed the IT network pivoted in any significant way to the OT environment. While SCADA/ICS attacks make for good scary stories, attackers aiming for OT effects are more sophisticated, and likely have more focused targeting.
- ✓ The ransomware on the IT infrastructure didn’t impact the fake factory – the spate of recent OT ransomware may be more likely due to OT/IT dependencies (e.g., billing systems) versus sophisticated ransomware gangs targeting the process engineering.
- ✓ The initial attack came almost two months after the honeypot was deployed, a surprisingly long period for a network with such glaring Internet-facing security vulnerabilities.



This work takes high interaction to the next level, not only building a virtual IT network, but also a full virtual SCADA factory, and fake company website – complete with corporate personas and realistic addresses.

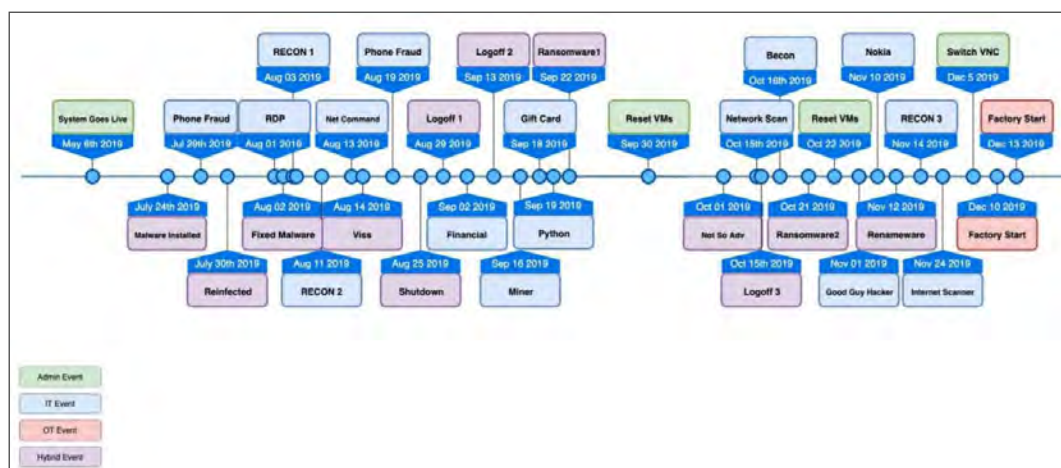


Figure 18: A timeline of malicious events on the fake factory network (both IT and OT) over the duration of ~7 months.

DO YOU SPEAK MY LANGUAGE? MAKING STATIC ANALYSIS ENGINES UNDERSTAND EACH OTHER

[View
Slides](#)

Authors: Ibrahim Mohamed and Manuel Fahndrich

This talk from Facebook security engineers highlighted the challenges of deploying static analysis at scale in modern applications. Complex microservice-based applications have components written in multiple languages. Detecting, say, SQL injection across these heterogeneous components means that multiple static analysis tools are needed to share and access taint information to be able to analyse the whole flow. Using the internals at Facebook (Hack/PHP, Python, and Java) as a motivation, a cross-repository database was built to store canonical API names that are called over RPC, and an enriching flow for both sources and sinks that allow a holistic trace to be generated for SQL injection or remote command execution across multiple repositories and languages. Finally this is evaluated at scale through a Facebook deployment, where each repository is periodically examined, as well as a comparison for each pull request. To minimise

false positives impacting development teams, only the highest confidence issues are automatically commented/ticketed. Lower confidence findings are sent to a security on-call first to minimise false positive pain.

TAKEAWAYS:

- ✓ With lightweight taint and source/sink information, it is possible to canonicalise across repositories and languages, finally bringing whole-application static analysis to the mainstream. Until this work, all major applications had black-box components written in another language or framework that reduced analysis performance, either increasing the false positives or decreasing the true positive findings.

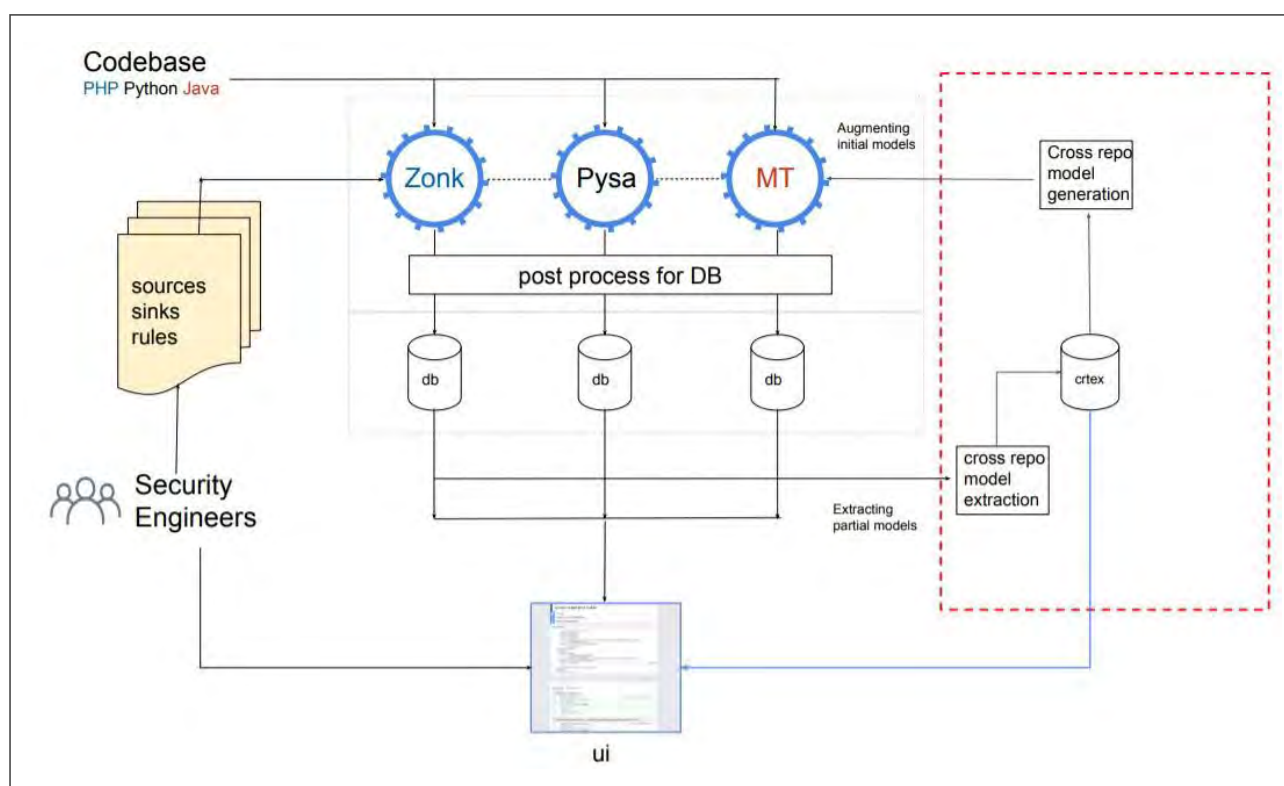


Figure 19: A figure showing the addition of cross-language and repository model generation for multi-language taint tracking.

PRACTICAL DEFENCES AGAINST ADVERSARIAL MACHINE LEARNING

[Watch Video](#)

Author: Ariel Herbert-Voss

Machine learning has received ample attention in recent years, specifically amongst academic researchers. This talk aims to cut through what-ifs, and presents experience from the trenches in “breaking ML systems deployed in real-world contexts”. Early in the talk, the author makes the striking statement that “adversarial ML [in real-life] is wildly different from academic research”. They highlight how academic research can have artificial constraints, and that they often focus on attacks which simply are not [yet] seen in the wild.

Observed in-the-wild attacks are broadly split into bad inputs and model leakage. Easy examples to understand for bad inputs are actions like placing stickers on a road to fool self-driving cars. For model leakage, they referenced an attack that extracted the model of Cylance’s malware classifier, in order to produce undetected malware offline. The author highlights that most of the real-world attacks seen have some financial motivation, for example manipulating recommendation engines to drive web traffic to certain online sellers.

The author then steps through a number of defences that have been effective against these real-world attacks. Some are extremely simple (such as implementing block-lists when input sources are known ahead of time). Others add a bit of complexity, relying on multiple signals instead of just one (e.g., two cameras reduce attacks against facial recognition systems significantly).

TAKEAWAYS:

- ✓ Machine learning is one area of security in which industry generally lags behind academia. It’s possible for the leading edge of academia to extend far beyond what’s currently useful, and a return to on-the-ground experience is refreshing.
- ✓ For folks deploying ML who are concerned about adversarial attacks, consider straight-forward defences before looking for complex solutions.

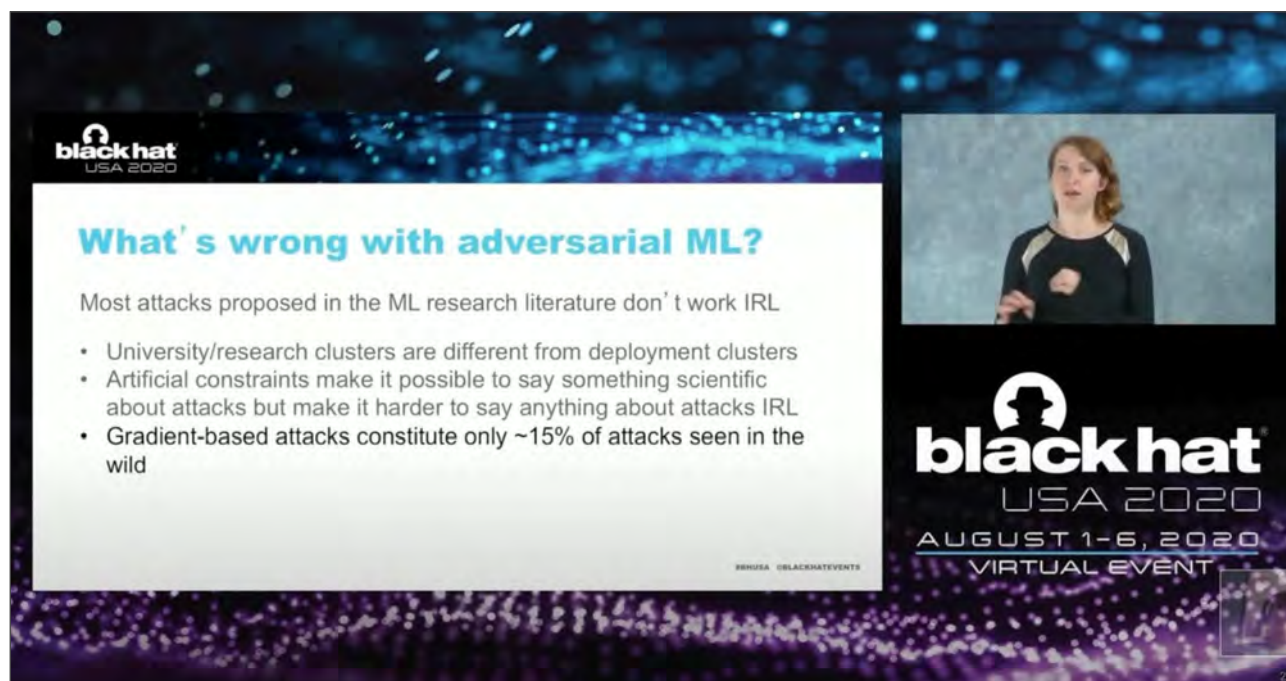


Figure 20: A summary of the differences between in-the-wild attacks and those studied in academic literature.

Nifty sundries

- ✓ *Remote Side-Channel Attacks on Anonymous Transactions*
- ✓ *An Observational Investigation of Reverse Engineers' Processes*
- ✓ *On the Feasibility of Automating Stock Market Manipulation*
- ✓ *IoT Skimmer: Energy Market Manipulation through High-Wattage IoT Botnets*
- ✓ *The Dark Age of Memory Corruption Mitigations in the Spectre Era*
- ✓ *Everything Old is New Again: Binary Security of WebAssembly*
- ✓ *ProxyLogon is Just the Tip of the Iceberg: A New Attack Surface on Microsoft Exchange Server!*

REMOTE SIDE-CHANNEL ATTACKS ON ANONYMOUS TRANSACTIONS

Read
Paper

Authors: Florian Tramer, Dan Boneh, and Kenneth G. Paterson

Cryptocurrencies present a fascinating environment for security and robustness research due to their built-in [oft-unintentional] bug bounties. Mistakes are an opportunity to earn money, regardless of the coin holder's knowledge or permission. Often the owner of some smart contract or wallet first becomes aware of a security issue after the contract or wallet has been drained.

Within the smart contract world, the notion of application bugs has already received wide attention. There are specialist smart contract auditing teams, and there are efforts to reduce the chance of mistakes through better implementation languages.

Orthogonal to cryptocurrency security research is work that aims to tackle privacy-focused cryptocurrencies such as Zcash and Monero. These aim to prevent third-parties from discovering who the sender and recipients are in transactions, as well as the amount being paid. While they can also suffer attacks against their transactional security, they must additionally consider attacks that compromise their privacy promises.

The lever for the attacks in this paper is not cryptography, but instead takes the form of network responses (both the time of the responses and the content). Since cryptocurrencies rely on large peer-to-peer networks, the authors observed that it's possible to remotely link supposedly anonymous transactions

to specific peers based on the timing and content of responses from the peers, for a variety of reasons.

For example, an attacker can encrypt a badly formatted message with some public key they suspect belongs to a particular peer. When the peer receives the message, if they hold the corresponding private key and decrypt the message, they'll see the broken message and respond with a rejection message. If they don't hold the corresponding private key, no such message is sent. In this way, the attacker can conclusively link keys to IP addresses – revealing a privacy breach.

They go on to describe additional attacks that show all the hallmarks of traditional network application hacking. Cryptocurrencies rely heavily on their networks; while these may be overlaid across lower-level carrier networks, the same kind of attacks we've seen before can be very effective as this work displays.

Takeaways:

- ✓ Cryptocurrencies are not just susceptible to money-stealing attacks. Privacy coins have a heavy burden to carry in terms of their privacy guarantees.
- ✓ Network attacks will also pop-up in overlay networks such as peer-to-peer networks, and this hasn't received nearly as much attention.

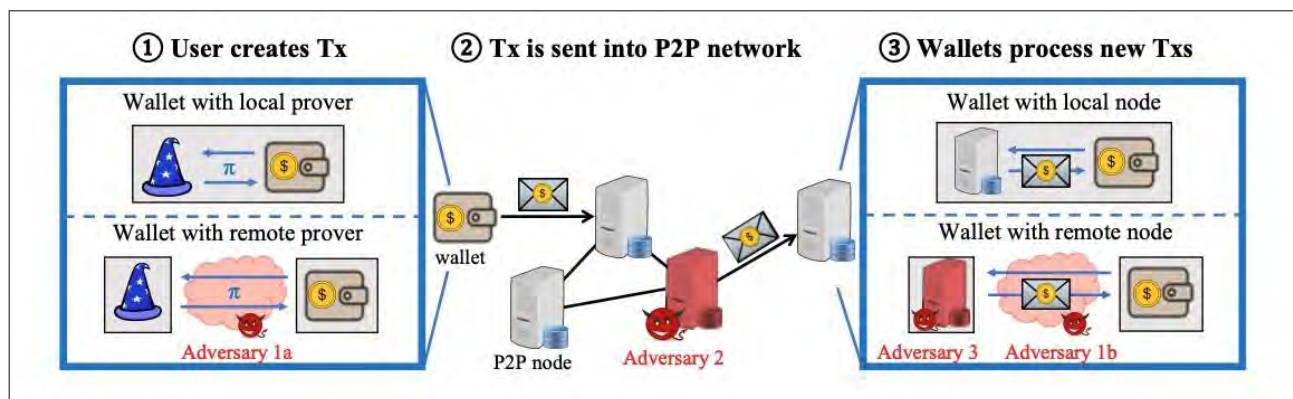


Figure 21: Side-channels in the anonymous transaction lifecycle.

AN OBSERVATIONAL INVESTIGATION OF REVERSE ENGINEERS' PROCESSES

[Read
Paper](#)

[Watch
Video](#)

Authors: Daniel Votipka, Seth Rabin, Kristopher Micinski, Jeffrey Foster, and Michelle Mazurek

These authors worked with a population of malware reversers and vulnerability researchers to study the processes used by these groups in deconstructing software. Despite a small population sample, an overarching process was identified that showed how different tools, techniques and approaches were combined into a high-level process flow that was relatively consistent across respondents. The first stage combines static analysis of strings, APIs/system-calls used, and executing the target to try to get a high-level view of the functionality present. Then the researcher or reverser's experience and intuition guide them to the program components warranting additional static analysis. Once specific portions are identified (~100 SLoC), dynamic analysis is reintroduced to verify assumptions, generate constraints, and understand how each component operates. Finally this knowledge is brought back to the larger-scale mental model of the target.

Despite the differing goals between reversers and vulnerability researchers, there was significant overlap between both populations. Future research is warranted to understand how the information gathered following this process is then converted into the specific output needed (i.e., target semantics or bugs), and how similar the intuitions are into which program components will yield the most fruitful results.

TAKEAWAYS:

- ✓ Very little focus has been applied to how reverse engineers or vulnerability researchers approach a large unknown target. While this work is interview based, and with a small sample size, it is interesting to see that practitioners with different backgrounds and training have similar methodologies.
- ✓ **Research efforts** have shown how combining human intelligence with automation have improved bug-finding results. Codifying human processes and intuition should enable additional improvements in mixed-initiative vulnerability research.
- ✓ The existence of this work highlights the maturity of these fields – a decade ago, doing university UX surveys on reverse engineering or vulnerability research would have been laughable, now it's worthy of a top-tier academic venue paper.
- ✓ Our field has an extreme paucity of epistemic research. This is a tentative step down what looks like a very long road.

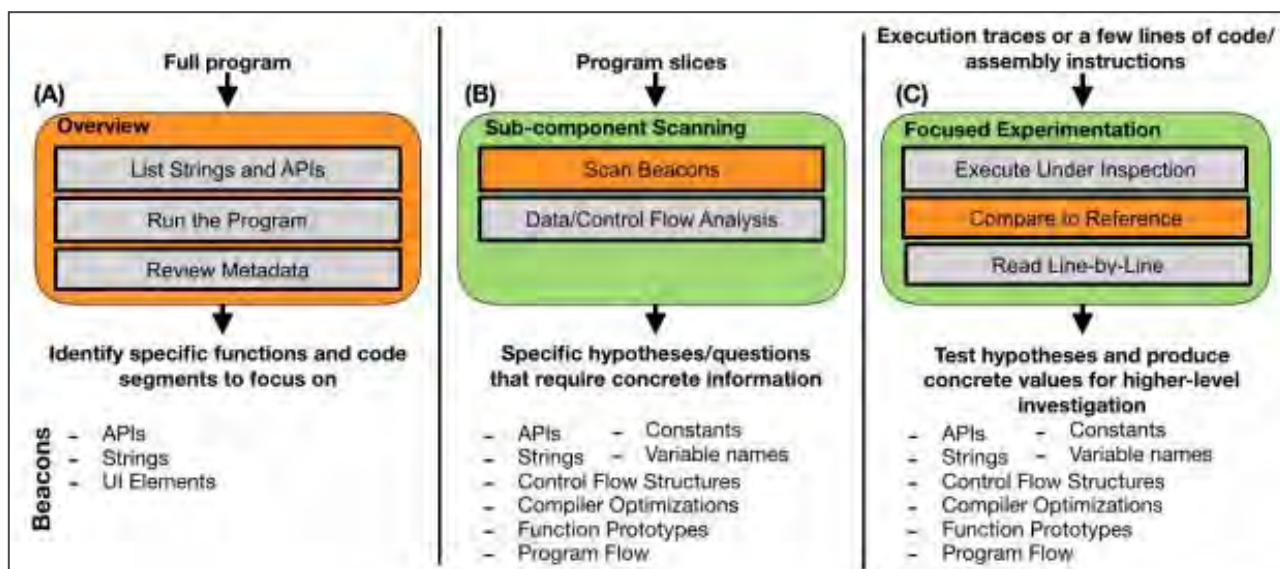


Figure 22: The high-level process flow identified across the surveyed population.



On the Feasibility of Automating Stock Market Manipulation

Read Paper

Authors: Carter Yagemann, Simon Chung, Erkam Uzun, Sai Ragam, Brendan Saltaformaggio, and Wenke Lee

Both of these works present methods to use a botnet for influence in parallel domains: the stock market and power market respectively. In their respective domains, they characterise the botnet scale needed (either in terms of spending power or wattage demand) to influence the market sufficiently for an attacker to make a profit.

In the stock market context, the first paper shows how a relatively small number of coordinated bots are needed (IBM shares as an example: four bots, each with \$5000 of spending power) to perform a layering attack that generates upward or downward momentum on a target stock to an attacker's benefit. Using the amplification present of other (ostensibly benign) high-frequency trading bots in the market to boost the swing initiated by a number of distributed orders expected to be cancelled prior to execution, a temporary swing can be created in a few minutes, allowing an attacker to execute at the peaks and troughs. With access to bots controlling ~1,000 brokerage accounts briefly over the course of a year, an estimated growth of \$100,000 to \$1,022,000 is feasible, whilst never reusing an account (minimising trading-pattern detection risk), and optionally using a percentage of the profits to keep the victim accounts at approximately the same value (minimising account compromise detection risk).

Applying a similar analysis to power markets, the second set of researchers built on previous work on using high-wattage IoT devices (e.g., an EV that can charge at certain times, or an AC connected to an IoT thermostat) to potentially cause instability on the grid. Instead of causing denial-of-service as demonstrated

IoT Skimmer: Energy Market Manipulation through High-Wattage IoT Botnets

View Slides

Authors: Tohid Shekari and Raheem Beyah

in past works, an analysis was performed on the scale of a high-wattage IoT botnet needed to either influence pricing (to enrich a conspiring power generation facility), or to simply cause economic damage by forcing utilities to use expensive reserve generators. This not only costs the consumers more in usage rates, but also has a higher financial impact on the operators of the generators (which are typically rarely used and expensive to operate). Fortunately, the work was unable to show sufficient availability of IoT botnets on high-wattage devices; the estimated 50,000 bots needed (in a specific geographic area) are unlikely to be easily available at present.

TAKEAWAYS:

- ✓ The rise of interconnected externalities will amplify attackers' ability to use artificially generated demand to shape markets for their benefits. While greater transparency into the market drivers would allow for better defences against malicious influence, it can cause privacy concerns – balancing security and privacy for the important (and increasingly diverse) markets in our lives is a policy challenge thus far unresolved.
- ✓ Attackers have shown increased sophistication in leveraging digital accesses for parallel benefits. Both papers show how markets can be weaponized by a nation-state in a manner other than DoS – combining market manipulation with targeted DoS of the checks and balances in these markets could trigger cascading failures, e.g., bank runs or rolling black-outs.

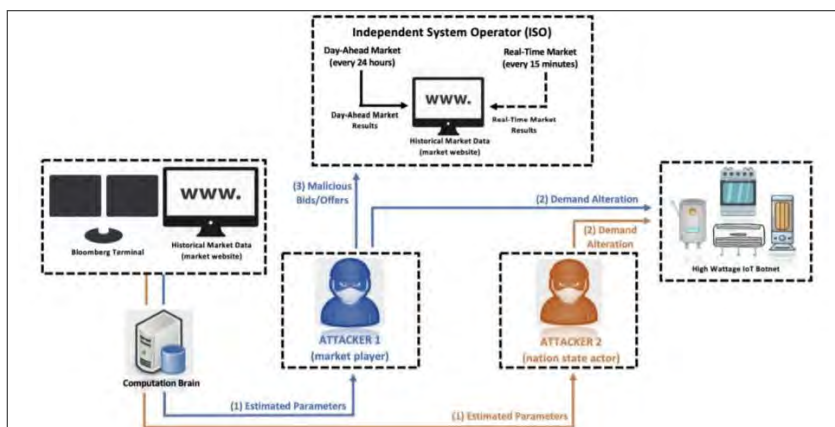


Figure 23: A figure depicting the two types of attackers examined in the power market scenario: a power provider looking to increase prices, and a nation state looking to cause financial harm.

THE DARK AGE OF MEMORY CORRUPTION MITIGATIONS IN THE SPECTRE ERA

[View
Slides](#)

Authors: Andrea Mambretti and Alexandra Sandulescu

This research looked at many of the existing speculative execution attacks that typically are described as a way to leak sensitive information – primarily a key or password – to further more conventional attacks. Using these primitives to “roll-back” the mitigations that have finally become mainstream allows for old-fashioned memory

corruption attacks to once again achieve RCE. The work looks at: leaking stack canaries, then using them in stack-based buffer overflow exploits, performing speculative-enabled ROP, and exploring other more modern CFI technologies for vulnerabilities.

The constraints on the speculative attacks mean they offer the most value when combined with traditional attacks. Using the primitives in a hybrid manner allows for greater chances of attack success and the ability to have impacts if more than side-channel information leakage typical for speculative attacks in isolation occurs.



Figure 24: Example of using speculative execution to leak canary values and bypass conventional exploit mitigations.

TAKEAWAYS:

- While speculative execution attacks have been presented as a number of individual primitives, it is important to step back to examine how these primitives can be used in an overall attack. Speculatively leaking canaries to bypass stack protections to enable RCE is a powerful chain, and there is no end in sight for novel micro-architectural attacks.

EVERYTHING OLD IS NEW AGAIN: BINARY SECURITY OF WEBASSEMBLY

[View
Slides](#)

[Read
Paper](#)

[Watch
Video](#)

Authors: Daniel Mehmman, Johannes Kinder, and Michael Pradel

This work reviewed the security model of WebAssembly, a compilation target for a number of unmanaged languages (e.g., C/C++) to run either on a client browser or back-end as a Node.js application. Despite strong claims of security by the WebAssembly designers, the researchers found that a number of exploit mitigations that have become part and parcel of modern execution environments were removed, allowing both old attacks and enabling new primitives. Memory layouts were contiguous, non-randomised, and combined both the heap and stack into a single region, allowing stack-to-heap overflows to manipulate the execution environment's state. By manipulating the

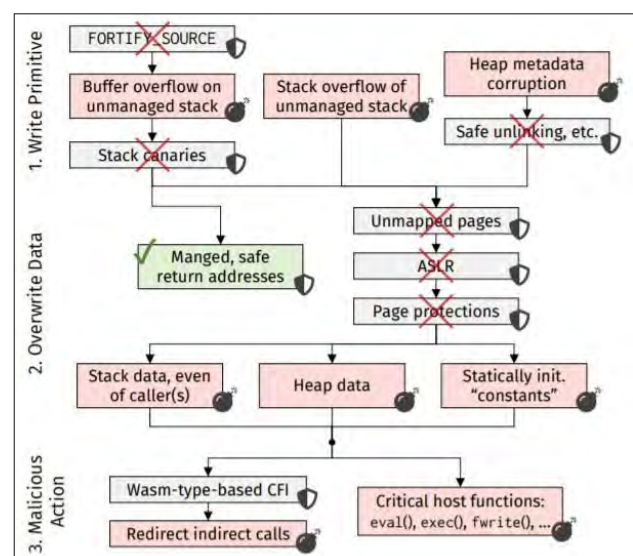


Figure 25: A figure showing attacks (represented by bomb icons) and missing defenses (shield icons) against WebAssembly environments.

state, calls to code execution functions (e.g., eval) could be controlled, and even host file-system access was possible.

While WebAssembly did ensure function return addresses were managed and safe (i.e., a shadow stack), other security improvements from the last decade were eliminated, reinstating vulnerabilities that would have been mitigated on native systems when compiled for WebAssembly runtimes.

TAKEAWAYS:

- ✓ Vulnerabilities exist in the space between a programmer's assumptions/mental model for how the environment operates and reality – rehosting an application in a markedly different environment is bound to expose novel exploitation possibilities.

PROXYLOGON IS JUST THE TIP OF THE ICEBERG: A NEW ATTACK SURFACE ON MICROSOFT EXCHANGE SERVER!

[View Slides](#)

Author: Orange Tsai

This recent work exploiting Microsoft Exchange Server is notable for several reasons. Exchange Server is a mature MTA (it's 25 years old!) and serves as the cornerstone to Microsoft's widely used email and calendaring solution. It's often publicly exposed to the Internet (the author mentions finding 400k instances online), and its integration into Active Directory is often very deep. Microsoft has invested heavily into their SDL for Exchange Server, including resources in secure design, coding and testing. Despite these investments, the author was involved in finding eight separate remotely exploitable bugs in Exchange Server over a period of six months. These exploits led to the automated HAFNIUM attacks against Exchange Servers.

What contributed to this outcome? Exchange Server added complexity with each additional major release, and part of this was to introduce the Client Access Service (CAS) in Exchange 2016. This is a multi-protocol handler which includes client access via POP3 / IMAP4, but importantly also HTTP/HTTPS. CAS acts as a proxy, carrying these client connections further on into the Exchange architecture. This work focuses exclusively on issues in the HTTP proxying from the CAS to the MTA back-end.

The issues relate to how two separate web servers and handlers inside Exchange Server interact with one another. Combining dynamic web application testing with decompiled source code revealed the issues. What's notable here, apart from Exchange Server shipping with unauthenticated remote code execution bugs for years, is that this is what the leading edge of real-world web exploitation looks like. Deep down inside Exchange, there's a secondary HTTP channel, and these attacks are targeting that. The dominance of HTTP as a messaging service knows no bounds.



Report Time	Name	CVE	Patch Time	Reported by
Jan 05, 2021	ProxyLogon	CVE-2021-26855	Mar 02, 2021	Orange Tsai, Volexity and MSTIC
Jan 05, 2021	ProxyLogon	CVE-2021-27065	Mar 02, 2021	Orange Tsai, Volexity and MSTIC
Jan 17, 2021	ProxyOracle	CVE-2021-31196	Jul 13, 2021	Orange Tsai
Jan 17, 2021	ProxyOracle	CVE-2021-31195	May 11, 2021	Orange Tsai
Apr 02, 2021	ProxyShell (PowerCat Bug)	CVE-2021-34473	Apr 13, 2021	Orange Tsai (working with Zix)
Apr 02, 2021	ProxyShell (PowerCat Bug)	CVE-2021-34523	Apr 13, 2021	Orange Tsai (working with Zix)
Apr 02, 2021	ProxyShell (PowerCat Bug)	CVE-2021-31207	May 11, 2021	Orange Tsai (working with Zix)
Jun 02, 2021	-	-	-	Orange Tsai

Figure 26: A table of the vulnerabilities discovered in a 6-month time period.

Despite this excellent work, Microsoft surprisingly refused to award any form of bounty, according to the author. Microsoft is so focused on moving customers to their Azure cloud services that they don't award bounties for on-premise Exchange Server vulnerabilities. This is concerning for on-premise customers; as there's even less incentive for bug finders to report to Microsoft.

TAKEAWAYS:

- ✓ Web application security seems stagnant with few new ideas, but the work shows that deep down in complex stacks there are HTTP attacks to be found.
- ✓ Exchange Server shipped with multiple RCEs in the default install.
- ✓ Microsoft not paying bounties for on-prem Exchange Server vulnerabilities means that there's an increased risk of vulnerabilities sold privately.

Conclusion

As the first issue of ThinkstScapes in some years, the aperture was widened to cover work that may have been missed during the COVID-19 driven format change to virtual conferences. While there was indeed a downward trend in the number of conferences and talks year-on-year as the pandemic and lock-downs hit, it remains to be seen what will happen as the world settles into our new normal.

FOUR THEMES, SHOWN THROUGH THE LAST YEAR OR SO OF WORK, WERE HIGHLIGHTED:

1. High-impact embedded research.
2. Complex systems and their interface weaknesses.
3. Real-world defence.
4. Novel works in classes of their own.

The next issue, in Q4 of 2021, will focus more deeply on the content released world-wide in the coming quarter as opposed to a survey of over a year of content.

